FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

1

RANDOM WALK SIMULATION USING AGENTPY

Student Name	Anonuevo, Kassandra Margarette F.
Section:	BSCSSE - AN43
Professor:	Tan, Crisola
Date:	October 9, 2026

I. BACKGROUND INFORMATION

A **random walk** is a mathematical process that describes a path consisting of a sequence of **random steps**. Random walks are used in many fields, such as physics, biology, economics, and computer science, to model systems that evolve randomly over time.

In this laboratory, we simulate multiple agents moving randomly on a grid using **AgentPy**, a Python library for **agent-based modeling**. Each agent represents an entity that can move one step at a time in a random direction. This simulation helps students understand:

- How **agents move and interact** in a grid-based environment.
- The concept of **bounded movement**, ensuring agents stay within the grid limits.
- What changing **parameters** like the number of agents, grid size, or number of steps affects the system.
- How **agent-based models** can be used to simulate real-world phenomena, such as **diffusion, crowd dynamics, or swarm behavior**.

II. TECHNICAL ASSESSMENT

RANDOM WALK SIMULATION USING AGENTPY

Objective

At the end of this laboratory activity, students are expected to:

1. To understand the concept of **random walks** in a grid environment.
2. To implement a **multi-agent simulation** using Python and AgentPy.
3. To visualize the **movement of agents** on a grid over time.

Materials Needed

- Computer with Python 3.x installed
- Libraries: agentpy, matplotlib
- Code editor (e.g., VSCode, PyCharm, or Jupyter Notebook)

General Instructions

1. This **Technical Assessment (TA4)** must be completed individually.
2. Follow the Task below.
3. Include clear comments in your code to describe its logic.
4. Each program should display the correct input, process, and output.
5. Ensure that your script displays clear and readable outputs in the Command Window.
6. Compile all Python codes and corresponding outputs into one PDF file.
7. Save it as **TA1_lastname.pdf**
   (Example: TA1_Patilano.pdf)

8. Submit the **python file** and **PDF** file.

Activity Instructions

Step 1: Run the Code

1. Open the provided Python code.

Random Walk Simulation (Console-Based)

```
import agentpy as ap
import random
import matplotlib.pyplot as plt
import matplotlib.animation as animation

# -------- Get user input from console --------
num_agents = int(input("Enter number of agents: "))
grid_size = int(input("Enter grid size (e.g., 10 for 10x10): "))
num_steps = int(input("Enter number of steps: "))

# -------- Agent Definition -----------
class RandomWalker(ap.Agent):
    def step(self):
        direction = random.choice([(1,0), (-1,0), (0,1), (0,-1)])
        x, y = self.position
        # Keep agents inside grid
        x = max(0, min(self.model.p.grid_size[0]-1, x + direction[0]))
        y = max(0, min(self.model.p.grid_size[1]-1, y + direction[1]))
        self.position = (x, y)

# -------- Model Definition -----------
class RandomWalkModel(ap.Model):
    def setup(self):
        # Create agents
        self.agents = ap.AgentList(self, self.p.agents, RandomWalker)

        # Initialize random positions
        for agent in self.agents:
            agent.position = (random.randint(0, self.p.grid_size[0]-1),
                              random.randint(0, self.p.grid_size[1]-1))

        # Create grid (optional)
        self.grid = ap.Grid(self, self.p.grid_size, torus=False)
        self.grid.add_agents(self.agents)

    def step(self):
        for agent in self.agents:
            agent.step()

# -------- Parameters from user input -----------
parameters = {
    'agents': num_agents,
    'grid_size': (grid_size, grid_size),
    'steps': num_steps
}

# -------- Run Model -----------
```

```
model = RandomWalkModel(parameters)
model.setup()

# -------- Interactive Animation -----------
fig, ax = plt.subplots()
ax.set_xlim(0, model.p.grid_size[0])
ax.set_ylim(0, model.p.grid_size[1])
ax.set_xticks(range(model.p.grid_size[0]+1))
ax.set_yticks(range(model.p.grid_size[1]+1))
ax.grid(True)

scat = ax.scatter([], [], s=200)  # s = size of agents

def update(frame):
    model.step()
    x = [agent.position[0] for agent in model.agents]
    y = [agent.position[1] for agent in model.agents]
    scat.set_offsets(list(zip(x, y)))
    return scat,

ani = animation.FuncAnimation(fig, update, frames=model.p.steps, blit=True, repeat=False)
plt.show()
```

2. Run the program in your terminal.
3. Enter the following values when prompted:
    - **Number of agents** (e.g., 5)
    - **Grid size** (e.g., 10 for a 10x10 grid)
    - **Number of steps** (e.g., 20)

These values control the number of agents, the size of the environment, and the simulation duration.

Step 2: Understand the Agent Behavior

1. Each agent is represented by a dot in the grid.
2. Agents move **randomly** in one of four directions: up, down, left, or right.
3. Agents **cannot leave the grid**; movement is restricted within the grid boundaries.

Step 3: Observe the Simulation

1. After running, a **grid with moving agents** is displayed.
2. Each time step updates the position of all agents.
3. Observe:
    - How agents spread across the grid over time
    - Whether agents cluster in certain areas or move freely

Step 4: Experiment with Parameters

- Repeat the simulation with **different values**:
    - **More agents** (e.g., 10 or 20)
    - **Smaller/larger grid sizes**
    - **Longer simulations** (e.g., 50 steps)
- Observe and record how the **agent movement patterns change**.

Laboratory Tasks

1. **Track agent paths**: Modify the code to **record and display the path of each agent** during the simulation.
2. **Change movement rules**: Make agents **avoid each other** or move in a **preferred direction** and observe the resulting patterns.
3. **Analyze final positions**: Record the **final positions** of all agents and analyze their **distribution**.
4. **Compare scenarios**: Run **two simulations with different parameters** and compare the movement patterns.

Modified (Updated Code - Laboratory Tasks)

```
import agentpy as ap
import random
import matplotlib.pyplot as plt
import matplotlib.animation as animation
from collections import Counter

# User input
num_agents = int(input("Enter number of agents: "))
grid_size = int(input("Enter grid size: "))
num_steps = int(input("Enter number of steps: "))


# Agent
class RandomWalker(ap.Agent):

    def setup(self):
        self.position = (
            random.randint(0, self.model.p.grid_size[0] - 1),
            random.randint(0, self.model.p.grid_size[1] - 1)
        )
        self.path = [self.position]

    def step(self):
        directions = [(1, 0), (-1, 0), (0, 1), (0, -1)]

        # Task 2: preferred right direction
        if self.model.p.preferred:
            directions = [(1, 0), (1, 0), (1, 0),
                          (-1, 0), (0, 1), (0, -1)]

        dx, dy = random.choice(directions)
        x, y = self.position
```

```
        x = max(0, min(self.model.p.grid_size[0] - 1, x + dx))
        y = max(0, min(self.model.p.grid_size[1] - 1, y + dy))

        self.position = (x, y)
        self.path.append(self.position)


# Model
class RandomWalkModel(ap.Model):

    def setup(self):
        self.agents = ap.AgentList(self, self.p.agents, RandomWalker)

    def step(self):
        for agent in self.agents:
            agent.step()


# Create model
def create_model(agents, size, steps, preferred=False):
    parameters = {
        "agents": agents,
        "grid_size": (size, size),
        "steps": steps,
        "preferred": preferred
    }

    model = RandomWalkModel(parameters)
    model.setup()
    return model


# Original animation
model = create_model(num_agents, grid_size, num_steps)

fig, ax = plt.subplots()
ax.set_xlim(0, grid_size)
ax.set_ylim(0, grid_size)
ax.set_xticks(range(grid_size + 1))
ax.set_yticks(range(grid_size + 1))
ax.grid(True)

scat = ax.scatter([], [], s=200)

def update(frame):
```

```
    if frame > 0:
        model.step()

    x = [a.position[0] for a in model.agents]
    y = [a.position[1] for a in model.agents]

    scat.set_offsets(list(zip(x, y)))
    ax.set_title(f"Random Walk - Step {frame}")

    return scat,

ani = animation.FuncAnimation(
    fig,
    update,
    frames=num_steps + 1,
    interval=500,
    repeat=False
)

plt.show()


# Task 1: Agent paths
plt.figure()

for i, agent in enumerate(model.agents):
    x, y = zip(*agent.path)
    plt.plot(x, y, marker=".", label=f"Agent {i + 1}")

plt.title("Task 1: Agent Paths")
plt.xlabel("X")
plt.ylabel("Y")
plt.grid(True)

if num_agents <= 10:
    plt.legend()

plt.show()


# Task 2: Preferred direction
preferred = create_model(
    num_agents, grid_size, num_steps, preferred=True
)
```

```
for _ in range(num_steps):
    preferred.step()

plt.figure()

for i, agent in enumerate(preferred.agents):
    x, y = zip(*agent.path)
    plt.plot(x, y, marker=".", label=f"Agent {i + 1}")

plt.title("Task 2: Preferred Right Direction")
plt.xlabel("X")
plt.ylabel("Y")
plt.grid(True)

if num_agents <= 10:
    plt.legend()

plt.show()


# Task 3: Final positions
print("\nTASK 3: FINAL POSITIONS")

positions = [a.position for a in model.agents]

for i, pos in enumerate(positions):
    print(f"Agent {i + 1}: {pos}")

print("Occupied cells:", len(Counter(positions)))

x, y = zip(*positions)

plt.figure()
plt.scatter(x, y, s=150)
plt.title("Task 3: Final Agent Positions")
plt.xlabel("X")
plt.ylabel("Y")
plt.grid(True)
plt.show()


# Task 4: Compare scenarios
scenario_a = create_model(5, 10, 20)
scenario_b = create_model(20, 10, 50)
```

```
for _ in range(20):
    scenario_a.step()

for _ in range(50):
    scenario_b.step()

fig, axes = plt.subplots(1, 2, figsize=(10, 4))

for ax, scenario, title in zip(
    axes,
    [scenario_a, scenario_b],
    ["Scenario A", "Scenario B"]
):

    for agent in scenario.agents:
        x, y = zip(*agent.path)
        ax.plot(x, y, marker=".", markersize=3)

    ax.set_title(
        f"{title}: {scenario.p.agents} Agents, "
        f"{scenario.p.steps} Steps"
    )
    ax.grid(True)

plt.tight_layout()
plt.show()

print("\nAll tasks completed.")
```

Step 5: Answer the Questions

1. What happens if the number of agents is **very high** compared to the grid size?
2. How does increasing the number of steps affect the **spread of agents**?
3. Do the agents ever move outside the grid? Why or why not?
4. How can this simulation be used to **model real-life phenomena** (e.g., diffusion, crowd movement)?

Answer the Questions ⇩

1. When the number of agents is very high compared to the grid size, the grid becomes crowded. More agents may occupy the same or nearby positions.

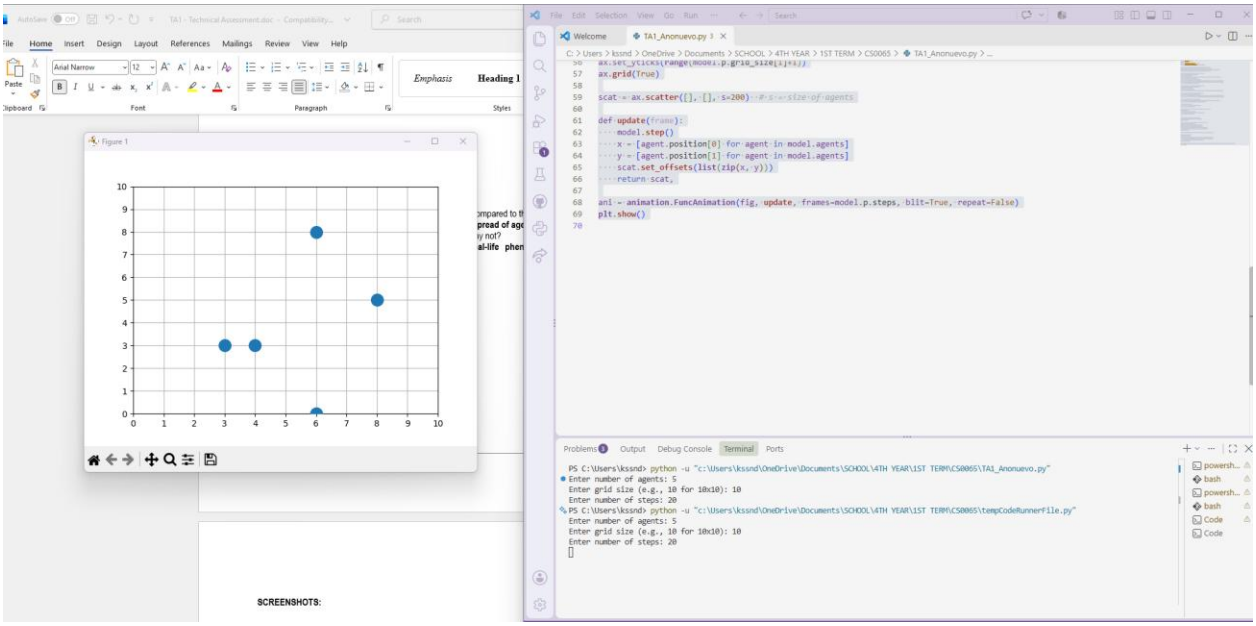
2. Increasing the number of steps gives the agents more chances to move, so they can travel farther and spread across more areas of the grid.

3. No. The agents do not move outside the grid because the code limits their positions to the grid boundaries.

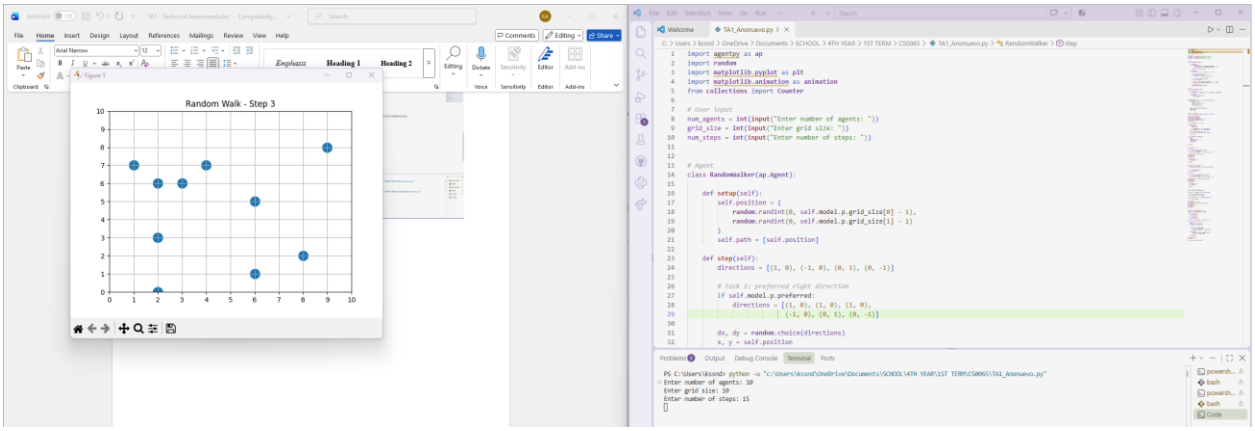
4. This simulation can model real-life situations where objects or people move randomly. For example, it can represent the spread of particles through diffusion or the movement and distribution of people in a crowded area.

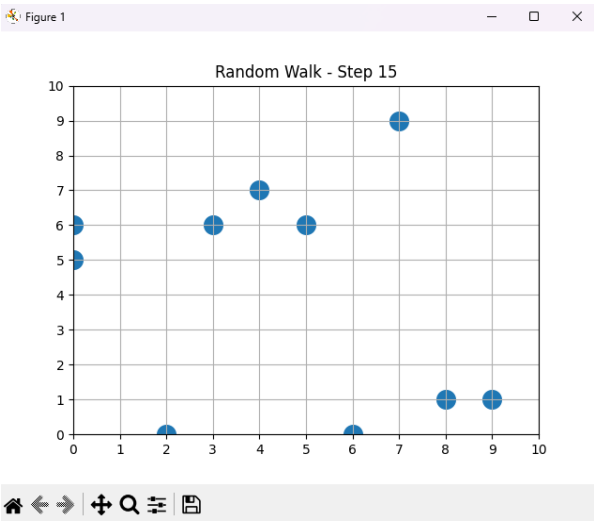
SCREENSHOTS:

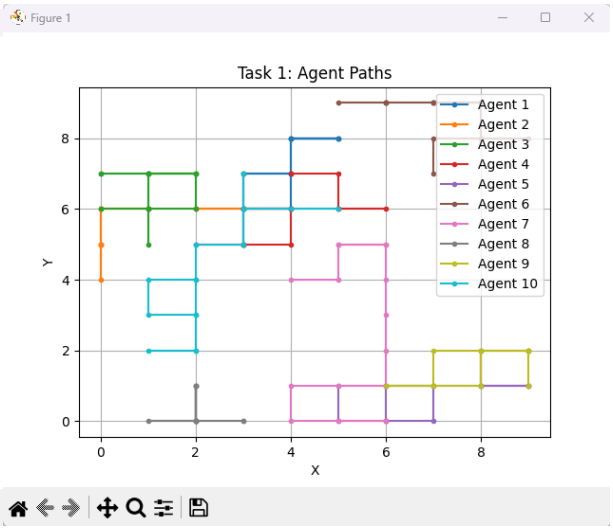
Given code:

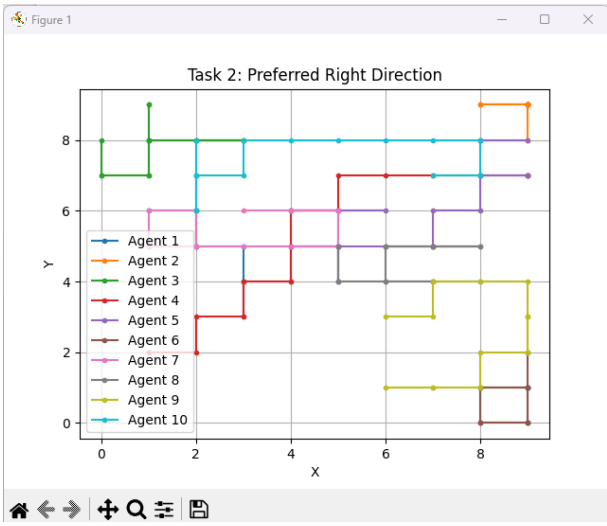


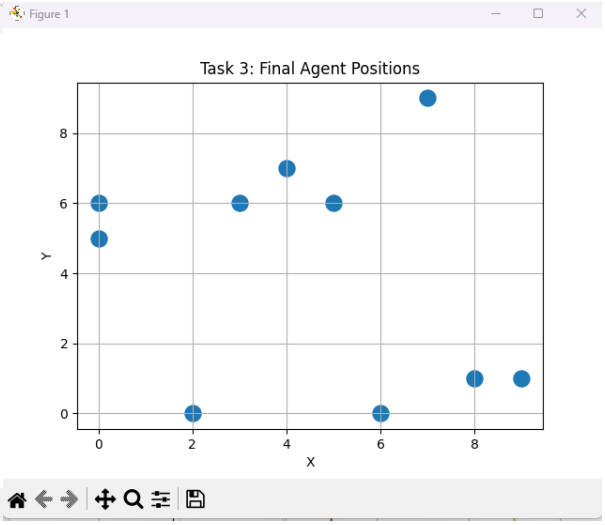
Modified Code:



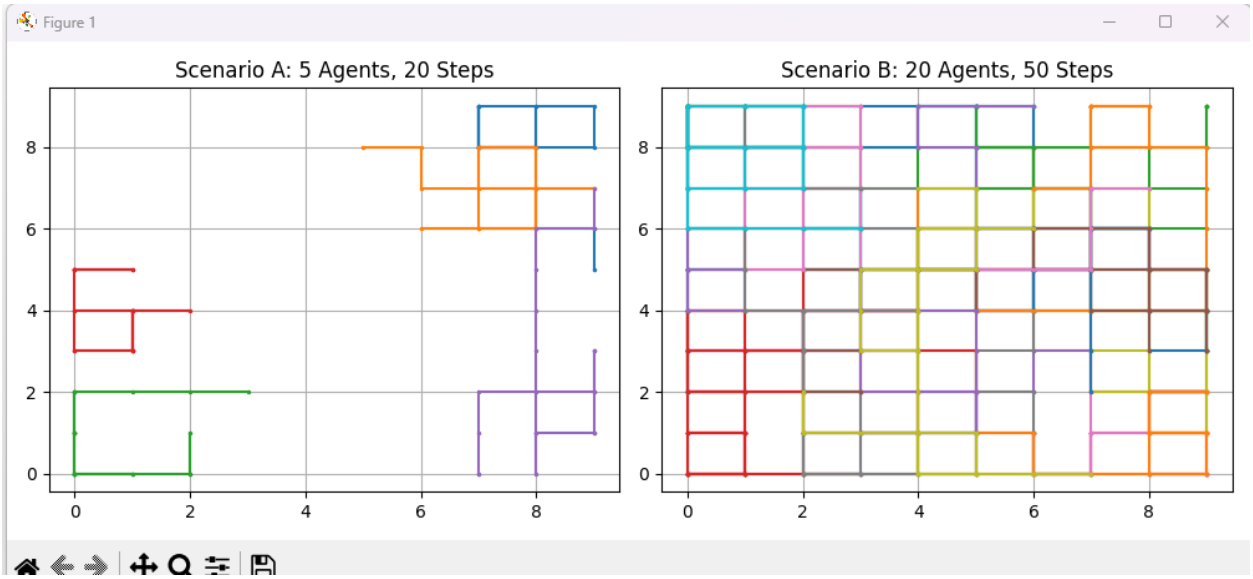









```
TASK 3: FINAL POSITIONS
Agent 1: (3, 6)
Agent 2: (0, 5)
Agent 3: (0, 6)
Agent 4: (4, 7)
Agent 5: (9, 1)
Agent 6: (7, 9)
Agent 7: (6, 0)
Agent 8: (2, 0)
Agent 9: (8, 1)
Agent 10: (5, 6)
Occupied cells: 10
```


Figure 1
Scenario A: 5 Agents, 20 Steps
Scenario B: 20 Agents, 50 Steps
0
2
4
6
8